



AI-Native Builder Playbook

Para Devs (E2E) — con mindset de producto

Basado en prácticas de ingenieros de Anthropic

Hardcore AI by 30X · Febrero 2026

AI-Native Builder Playbook para Devs (E2E) — con mindset de producto

El equipo de **Hardcore AI by 30X** preparó este playbook como una síntesis operativa de prácticas compartidas públicamente por ingenieros y líderes técnicos de Anthropic — Alex Albert, Zack Witten, Alexander Bricken, Joe Bayley, Mahesh Murag, Barry Zhang, John Welsh, Theodora Chu, Boris Cherny, Katelyn Lesse y Thariq Shihpar— en charlas y workshops sobre Claude, agentic coding, MCP, tooling y calidad en producción, y está adaptado para **desarrolladores de software** que construyen **productos end-to-end con IA** (copilots, agentes, RAG + tools, automatización) buscando resultados:

- **útiles** (mueve métricas de producto),
 - **confiables** (no se rompe al tercer usuario),
 - **operables** (se puede monitorear, depurar y mejorar),
 - **seguros** (scopes, auditoría, límites),
 - **mantenibles** (no se vuelve deuda técnica con esteroides).
-

Cómo usar este documento

- Si estás en 0→1: empieza en **1 (Producto)** → **2 (Arquitectura)** → **3 (Loop agente)** → **4 (Prompt/Context)** → **5 (Tools)** → **8 (Evals)**.
 - Si ya hay producción y duele: ve directo a **8 (Evals)** → **9 (Observabilidad)** → **10 (Seguridad)** → **11 (Enterprise/MCP/Gateway)**.
 - Si estás construyendo plataforma interna: enfócate en **5 (Tools)** → **6 (Skills)** → **11 (MCP/Gateway)** → **9 (Ops)**.
-

1) Producto primero: la IA no es una feature, es un flujo

1.1 Define el job-to-be-done (JTBD) y el “momento wow”

Antes de pensar en prompts o modelos, responde:

- ¿Qué **resultado** quiere el usuario?
 - ¿Qué se vuelve **más rápido / mejor / barato / más confiable**?
 - ¿Cómo se mide el éxito? (p.ej., tiempo a resolución, calidad, tasa de retrabajo, conversión)
-

1.2 Roadmap realista: 3 niveles de valor

1) **Productividad “table-stakes”** (valor rápido) 2) **Automatización de flujos críticos** (ventaja operativa) 3) **Secret sauce** (diferenciación real)

Regla: no intentes saltar al nivel 3 sin plataforma: terminas con demos frágiles.

1.3 Decide “qué no debe hacer” el modelo

Producto + riesgo definen límites:

- qué requiere confirmación
 - qué es autopilot
 - qué es solo sugerencia
-

2) Arquitectura mínima de un producto AI E2E

Piensa tu app como un **sistema**, no como “un chat”:

1) **UX**: input → progreso → confirmaciones → errores → “undo”. 2) **Orquestación (workflow)**: pasos, tool calls, reintentos, validaciones. 3) **Contexto**: retrieval, memoria, estado de sesión, reglas. 4) **Modelo**: razonamiento + generación. 5) **Tools/Acciones**: APIs, DB, servicios, búsqueda, ejecución controlada. 6) **Evals + Observabilidad**: datasets, métricas, trazas, costo, latencia. 7) **Seguridad**: permisos, scopes, sanitización, auditoría, límites.

2.1 Principio central: software grueso, modelo delgado

Todo lo determinista va en código:

- validaciones
- cálculos
- reglas
- formatos
- permisos

El modelo se usa para:

- lenguaje
 - síntesis
 - decisiones bajo ambigüedad
 - generación creativa
-

3) El loop de desarrollo con agentes: plan → act → check → fix

En agentic coding, tu rol cambia: **diseñas el bucle**, no micro-escribes cada línea.

3.1 Loop mínimo (sin esto, el agente es humo)

1) **Plan**: qué va a hacer y por qué (3–7 bullets) 2) **Act**: cambios pequeños (diffs manejables) 3) **Check**: señales objetivas (tests/build/lint/typecheck) 4) **Fix**: corrige con evidencia

Si no hay un “oracle” (tests/checks/rúbricas), el agente inventa.

3.2 Plan mode como control de seguridad y velocidad

Para tareas complejas:

- pide plan
- revisa
- ejecuta

Actúas como Tech Lead: apruebas estrategia, no commits aleatorios.

3.3 Iterar contra un target determinista

Targets típicos:

- `npm test` / `pytest`
- `tsc` / `mypy`
- `lint` / `format`
- build / compile
- golden files

Regla: cada iteración termina en una señal objetiva de “mejoró / empeoró”.

4) Prompt & Context Engineering (dev-first)

4.1 Contexto mínimo suficiente

Tu contexto debe ser:

- **relevante**
- **ordenado**
- **reciente**

Evita dumps masivos; usa IDs/refs y archivos de contexto.

4.2 Instrucciones específicas (no vibes)

No digas “sé conciso”. Di:

- “Devuelve 3 bullets, máximo 18 palabras cada uno.”
 - “Devuelve JSON con claves X,Y,Z.”
-

4.3 La salida imita la entrada

Si quieres tono/estructura, **muéstralo** con 1–2 ejemplos cortos.

4.4 Context as files

En vez de prompts gigantes, crea archivos:

- **AGENTS.md** (reglas)
- **ARCH.md** (arquitectura)
- **RUNBOOK.md** (operación)
- **DECISIONS.md** (decisiones)

El agente los lee cuando lo necesita.

4.5 Plantilla de prompt (lista para pegar)

ROL:

Eres un ingeniero de software senior con mindset de producto.

OBJETIVO (JTBD):

{qué quiere lograr el usuario}

CONTEXTO:

- Producto: {producto}
- Usuario: {perfil}
- Restricciones: {compliance, tiempos, stack}
- Fuentes: {docs, tickets, logs, PRDs}

TAREA:

{una frase clara}

FORMATO DE SALIDA:

{JSON/Markdown/etc. con estructura exacta}

CRITERIOS DE ÉXITO:

- {criterio 1}
- {criterio 2}

EJEMPLO:

Input: ...

Output: ...

5) Tools: del “chat” a “hacer cosas” sin romper producción

El 80% de los sistemas AI E2E “serios” se definen por su **capa de herramientas**.

5.1 Diseño de tools (agent-friendly)

Una tool buena:

- nombre claro
 - propósito explícito (qué hace / qué NO hace)
 - schemas estrictos
 - outputs estructurados
 - errores accionables
 - límites (rate, tamaño, timeout)
-

5.2 Tool Spec (plantilla)

- **Nombre**
 - **Propósito** (1 frase)
 - **Inputs** (schema + ejemplos)
 - **Outputs** (schema + ejemplos)
 - **Errores** (códigos + handling)
 - **Side-effects**
 - **Permisos**
 - **Límites**
 - **Observabilidad**
-

5.3 Permisos por tarea (no global)

No hagas: “todas las tools para todo”.

Haz:

- esta tarea solo puede **Read/Edit/Bash**
- esta otra solo puede **Search/Fetch**

Reduce errores y mejora seguridad.

5.4 Hooks: donde metes ingeniería real

- **Pre-tool hook:** valida args, aplica policy, redacta PII
 - **Post-tool hook:** valida outputs, cachea, loggea métricas
 - **Pre-commit hook:** exige tests/lint/format
-

5.5 Ejecución de código y tools riesgosas

Trátalas como un servicio controlado:

- sandbox
 - allowlists
 - límites de CPU/memoria/tiempo
 - auditoría
 - “dry run” antes de “apply”
-

6) Skills: modularidad que evita agentes monolíticos

6.1 Anti-pattern: “un súper agente que hace todo”

Terminas con:

- prompts gigantes
- comportamiento impredecible
- mezcla de dominio + ejecución
- onboarding lento
- deuda técnica acelerada

6.2 Patrón ganador: harness estable + skills versionadas

- **Harness / loop** (estable)
- **Skills** (conocimiento procedimental modular)

Piensa:

- Modelo = CPU
- Harness = OS
- Skills = Apps

6.3 Estructura de skill (lista para repo)

```
/skills
/pr-review
README.md
instruction.md
rubric.md
examples/
good.md
bad.md
prompts/
system.txt
worker.txt
scripts/
run_checks.sh
metadata.json
```

6.4 Skill Contract (lo mínimo exigible)

- Inputs
 - Outputs
 - Tools requeridas
 - Criterios de éxito
 - Casos donde NO aplica
-

7) Paralelización y multi-agente (sin caos)

7.1 Paraleliza investigación más que edición masiva

Buen paralelo:

- análisis → tests → docs → refactor aislado

Riesgoso:

- dos agentes editando la misma capa/archivos
-

7.2 Protocolo mínimo de coordinación

- cada agente produce un plan corto
 - diffs pequeños
 - un “lead agent” integra
 - suite final obligatoria
-

8) Calidad: evals, pruebas y regresión (la diferencia entre demo y producto)

8.1 Evals por jobs (no por prompts sueltos)

Ejemplos de jobs:

- “Responder ticket con reproducción + fix + release note”
- “Generar PR con tests”
- “Analizar documento y extraer riesgos”

8.2 Qué medir (en agentes)

1) planificación (plan útil) 2) tool selection (tool correcta) 3) tool arguments (parámetros correctos) 4) grounding (no inventa) 5) output (formato, tono, completitud)

8.3 Regresión automática

- golden set (20–100 casos)
- ejecutar por cada cambio de prompt/modelo
- diff de outputs
- tracking de fallos

8.4 “Prompt unit tests”

- casos típicos + edge cases
 - formatos estrictos
 - rúbrica simple
-

9) Observabilidad y operación (LLMOps para devs)

9.1 Métricas mínimas

- latencia p50/p95
 - costo por job
 - tokens input/output
 - tasa de fallos por tool
 - reintentos
 - outputs fuera de formato
 - incidentes (seguridad, data exposure)
-

9.2 Trazas mínimas por request

- prompt final
 - contexto usado (IDs, no dumps gigantes)
 - tool calls (nombre, args, duración, status)
 - output final
-

9.3 Rollout y resiliencia

- feature flags
 - canaries
 - rollback rápido
 - degradación (fallback)
-

9.4 Runbook (mínimo)

- cómo detectar incidentes
 - cómo mitigar
 - cómo hacer postmortem
 - cómo actualizar eval suite
-

10) Seguridad: Swiss-cheese defense (capas, no promesas)

No hay bala de plata. Defensa por capas:

- policies
- permisos del harness
- sandbox
- allowlists
- auditoría
- rate limits
- revisión humana (alto riesgo)

10.1 Acciones que casi siempre requieren confirmación

- cambios en auth/roles/permisos
 - comandos destructivos
 - cambios en infraestructura
 - borrados/movimientos masivos del repo
-

11) MCP y patrones enterprise (cuando escalas integraciones)

11.1 Cuándo usar MCP

Cuando:

- tienes muchas integraciones/tools
- varios equipos consumen las mismas capacidades
- necesitas contratos estables + seguridad + observabilidad

11.2 Heurísticas de tool-use robusto

- descubrir tools bajo demanda (no cargar todo)
- orquestación programática (loops/condicionales en código)
- ejemplos por tool (más que solo schema)

11.3 Patrón Gateway

Cuando creces, centraliza:

- Auth (SSO)
- policy/scopes
- rate limits
- auditoría
- routing
- catálogo/versiones

Beneficio: cada equipo deja de reinventar seguridad y observabilidad.

Checklists finales

Shipping checklist (MVP serio)

- JTBD + métrica definidos
 - Modelo delgado; lógica determinista en código
 - Prompt estructurado + ejemplos + formato
 - Tools con schemas + límites + errores accionables
 - Suite mínima de eval + regresión
 - Observabilidad (trazas/costo/latencia)
 - Fallbacks + rollout seguro
-

Enterprise-ready checklist

- scopes mínimos + approvals en alto riesgo
 - auditoría end-to-end
 - rate limits + quotas
 - runbook + postmortems
 - MCP/Gateway si hay múltiples equipos/tools
-

Cierre

Tu ventaja como builder no es “tener un modelo”. Es tener:

1) **un loop** que converge (plan→act→check→fix), 2) **una plataforma** (tools, permisos, observabilidad), 3) **conocimiento modular** (skills versionadas), 4) **calidad operable** (evals + regresión + runbooks).